

FIAP · NAVEGANDO PELO MUNDO DEVOPS

Cidades ESG Inteligentes

EnergiaApi — Desafio DevOps: CI/CD, containerização e orquestração

INTEGRANTES

Henrique Cole Fernandes — RM559957

GRUPO

Grupo 15

APLICAÇÃO BASE

ASP.NET Core 8 (C#) — Webservice de eficiência energética

REPOSITÓRIO

github.com/<usuario>/EnergiaApi

1. Visão geral do projeto

A **EnergiaApi** é um webservice RESTful de eficiência energética e sustentabilidade (ESG) desenvolvido em ASP.NET Core 8, reaproveitado dos desafios anteriores da trilha. A aplicação monitora o consumo de energia de equipamentos urbanos, gera alertas automáticos quando o consumo excede o limite configurado e produz relatórios agregados — a base de dados para decisões de eficiência energética de uma cidade inteligente.

Neste desafio a aplicação foi adaptada para simular um ambiente de produção real: o ciclo de vida inteiro (build, teste, empacotamento e entrega em dois ambientes) passou a ser automatizado, e a configuração foi externalizada para que a mesma imagem rode em desenvolvimento, homologação e produção.

Endpoints

Método	Rota	Descrição	JWT
POST	/api/auth/login	Autentica e devolve o token JWT	—
GET	/api/consumos	Lista paginada de consumos	—
POST	/api/leituras	Registra leitura e dispara alerta automático	Sim
GET	/api/relatorios/equipamento/{id}	Relatório agregado por equipamento	—
GET	/api/alertas	Lista paginada de alertas	Sim
GET	/health	Status, ambiente e versão da instância — criado para o pipeline	—

O que foi acrescentado na fase DevOps

Artefato	Função
.github/workflows/ci-cd.yml	Pipeline de CI/CD com 4 jobs: build/testes, imagem, deploy staging, deploy produção
EnergiaApi.Api/Dockerfile	Imagem multi-stage, usuário sem privilégios, configuração por variável de ambiente
docker-compose.yml	Orquestração de API + SQL Server, totalmente parametrizada por variáveis
.env.example / .env.staging / .env.production	Configuração por ambiente, fora do código
scripts/smoke-test.sh	Validação automatizada de cada deploy (teste de fumaça)
/health na API	Ponto de verificação usado pelo pipeline e pelas evidências

Estrutura do repositório

```
EnergiaApi/  
├── .github/workflows/ci-cd.yml    Pipeline de CI/CD  
├── EnergiaApi.Api/              API (Models, ViewModels, Data, Services, Controllers, Migrations)  
│   └── Dockerfile                Imagem multi-stage da aplicação  
├── EnergiaApi.Tests/            6 testes de integração xUnit  
├── frontend/                    Aplicação React (Vite + TypeScript)  
├── postman/                      Coleção e environment do Postman  
├── scripts/smoke-test.sh         Validação de cada deploy  
├── docker-compose.yml            Orquestração (volume, rede, variáveis)  
├── .env.example .env.staging .env.production  
└── README.md
```

2. Descrição do pipeline

Ferramenta: GitHub Actions. Escolhida por ficar no mesmo lugar do código-fonte — sem servidor de build para manter, com runners hospedados que já trazem Docker e Compose instalados, e com o recurso de *Environments*, que entrega separação de segredos e aprovação manual sem nenhuma ferramenta extra.

Registro de imagens: GitHub Container Registry (GHCR), também integrado ao repositório e autenticado com o `GITHUB_TOKEN` do próprio run.

Gatilhos: `push` na `main` executa o pipeline completo; `pull_request` executa apenas build e testes, barrando código quebrado antes do merge; `workflow_dispatch` permite execução manual.

Etapas



#	Job	O que faz	Lógica / por quê
1	build-test	dotnet restore, dotnet build -c Release e dotnet test com os 6 testes de integração. O relatório .trx sobe como artefato do run.	Portão de qualidade: nenhum job seguinte começa se a compilação ou um teste falhar.
2	image	Constrói a imagem pelo Dockerfile e publica no GHCR com as tags :<sha-curto> e :latest, com cache de camadas do Actions (type=gha).	Produz um artefato imutável identificado pelo commit , que será promovido pelos dois ambientes.
3	deploy-staging	Login no GHCR, docker compose --env-file .env.staging -p energia-staging up -d --no-build e scripts/smoke-test.sh contra a porta 8081. Publica os logs da aplicação no fim do job.	Valida a imagem em um ambiente equivalente ao de produção antes de qualquer exposição ao usuário final.
4	deploy-production	Depende do sucesso de staging (needs: deploy-staging) e sobe a mesma imagem com .env.production na porta 8082, seguido do smoke test.	Promoção de artefato: produção recebe exatamente o binário aprovado, sem rebuild — elimina a classe de bug “funcionava em staging”.

Testes automatizados

A suíte xUnit sobe a API em memória com `WebApplicationFactory` e banco EF Core InMemory, cobrindo os 5 endpoints de negócio e o `/health`:

```
$ dotnet test
Passed! - Failed: 0, Passed: 6, Skipped: 0, Total: 6, Duration: 586 ms
```

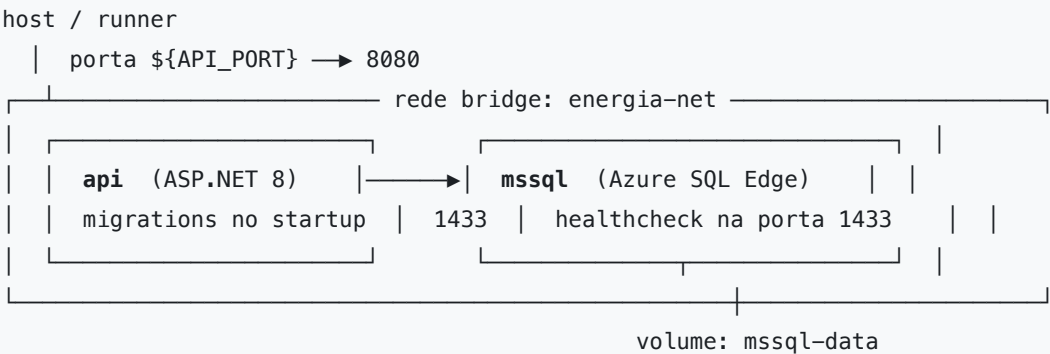
Além disso, **cada deploy é verificado pelo smoke test**, que exercita o ambiente já publicado: aguarda o `/health`, autentica em `/api/auth/login`, consulta `/api/consumos` (provando que a API alcança o banco e que as migrations rodaram) e chama a rota protegida `/api/alertas` com o token JWT. Qualquer passo que falhe derruba o job e o deploy é considerado quebrado.

Ambientes e segredos

- **GitHub Environments** `staging` e `production`: cada um com sua URL exibida na tela do run e seus próprios segredos.
- **Segredos por ambiente**: `DB_PASSWORD` e `JWT_KEY` são injetados como variáveis de ambiente no container — nunca ficam no código nem na imagem.
- **Aprovação manual em produção**: configurada em *Settings* → *Environments* → *production* → *Required reviewers*, o que transforma o último job em um deploy sob aprovação.
- **Fallback**: sem os secrets configurados, o workflow usa os valores dos arquivos `.env.*` e roda de ponta a ponta em qualquer fork, sem configuração prévia.

3. Docker: arquitetura e imagem

Arquitetura do ambiente



Dockerfile

```
# --- Estagio 1: build/publish (SDK, imagem grande, descartada no final) ---
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

# Copia so o .csproj primeiro: enquanto as dependencias nao mudam,
# o Docker reaproveita a camada de restore em cache.
COPY ["EnergiaApi.Api/EnergiaApi.Api.csproj", "EnergiaApi.Api/"]
RUN dotnet restore "EnergiaApi.Api/EnergiaApi.Api.csproj"

COPY . .
RUN dotnet publish "EnergiaApi.Api/EnergiaApi.Api.csproj" \
    -c Release --no-restore -o /app/publish

# --- Estagio 2: runtime (so o ASP.NET Core, sem SDK nem codigo-fonte) ---
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app
COPY --from=build /app/publish .

ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080

# Roda como usuario sem privilegios (APP_UID=1654 ja vem definido na imagem base).
USER $APP_UID

ENTRYPOINT ["dotnet", "EnergiaApi.Api.dll"]
```

Estratégia	Implementação	Ganho
Multi-stage build	Estágio <code>build</code> com SDK; estágio <code>final</code> só com o runtime	A imagem entregue não carrega SDK, compilador nem código-fonte: menos peso e menor superfície de ataque
Cache de camadas	<code>COPY</code> do <code>.csproj</code> + <code>restore</code> antes do código	Mudar código não re-baixa pacotes NuGet; no pipeline o cache é reaproveitado via <code>type=gha</code>

Usuário sem privilégios	<code>USER \$APP_UID</code>	O processo não roda como root dentro do container
<code>.dockerignore</code>	Ignora <code>bin</code> , <code>obj</code> , <code>node_modules</code> , <code>frontend</code>	Contexto de build enxuto e reproduzível
Configuração externalizada	Nenhuma senha ou connection string dentro da imagem	A mesma imagem roda nos três ambientes — só o <code>--env-file</code> muda

Orquestração: os três recursos exigidos

Recurso	Onde	Para que
Volume	<code>mssql-data:/var/opt/mssql</code>	Os dados do banco sobrevivem ao <code>docker compose down</code> e à recriação do container
Rede	<code>energia-net</code> (bridge dedicada)	API e banco se enxergam pelo nome do serviço (<code>Server=mssql,1433</code>) e ficam isolados de outras stacks
Variáveis de ambiente	<code>.env</code> , <code>.env.staging</code> , <code>.env.production</code>	Porta, nome do banco, <code>ASPNETCORE_ENVIRONMENT</code> , senha e chave JWT fora do código

`depends_on: condition: service_healthy` garante a ordem de subida: a API só inicia depois que o SQL Server responde na 1433, o que evita o erro clássico de migration em container que sobe mais rápido que o banco.

Comandos utilizados

```
# ambiente local
cp .env.example .env
docker compose up -d --build
./scripts/smoke-test.sh http://localhost:8080

# staging (porta 8081, banco 14331)
docker compose --env-file .env.staging -p energia-staging up -d --build
./scripts/smoke-test.sh http://localhost:8081

# producao (porta 8082, banco 14332)
docker compose --env-file .env.production -p energia-prod up -d --build
./scripts/smoke-test.sh http://localhost:8082

# operacao
docker compose logs -f api
docker compose ps
docker compose down -v
```

Imagem criada

```
$ docker images energiaapi
IMAGE          ID                DISK USAGE  CONTENT SIZE
energiaapi:local  ede2cce21658      375MB       94.9MB
```

No pipeline a mesma imagem é publicada no GHCR como `ghcr.io/<usuario>/energiaapi:<sha-curto>` e `:latest`.

4. Evidências de execução

Abaixo, as saídas reais de terminal capturadas na execução local (arquivo completo em `docs/evidencias-locais.txt`). Os espaços marcados devem receber as capturas de tela do pipeline no GitHub Actions.

4.1 Build e testes automatizados

```
$ dotnet test
Test run for EnergiaApi.Tests/bin/Debug/net8.0/EnergiaApi.Tests.dll (.NETCoreApp,Version=v8.0)
A total of 1 test files matched the specified pattern.

Passed! - Failed: 0, Passed: 6, Skipped: 0, Total: 6, Duration: 586 ms
```

Print 1 — Pipeline completo

Aba Actions com os 4 jobs verdes
`docs/prints/01-pipeline.png`

Print 2 — Job build-test

Log do build em Release e do `dotnet test` com Passed: 6
`docs/prints/02-build.png` · `docs/prints/03-testes.png`

Print 3 — Imagem no GHCR

Aba Packages do repositório com as tags publicadas
`docs/prints/04-ghcr.png`

4.2 Deploy em staging e produção

```
$ docker ps --filter name=energia --format "table {{.Names}}\t{{.Ports}}\t{{.Status}}"
NAMES                                PORTS                                STATUS
energia-prod-api-1                   0.0.0.0:8082->8080/tcp              Up 22 seconds
energia-prod-mssql-1                 0.0.0.0:14332->1433/tcp             Up 27 seconds (healthy)
energia-staging-api-1                0.0.0.0:8081->8080/tcp              Up 55 seconds
energia-staging-mssql-1              0.0.0.0:14331->1433/tcp             Up About a minute (healthy)

$ docker volume ls --filter name=energia
local      energia-prod_mssql-data
local      energia-staging_mssql-data

$ docker network ls --filter name=energia
energia-prod_energia-net    bridge    local
energia-staging_energia-net bridge    local
```

Os dois ambientes rodando ao mesmo tempo, cada um com seu container, sua rede e seu volume — resultado do `-p` (project name) combinado com o `--env-file`.

4.3 Smoke test — STAGING

```
$ ./scripts/smoke-test.sh http://localhost:8081
==> Aguardando http://localhost:8081/health (timeout 180s)
    API respondeu em 0s
==> 1/4 GET /health
{"status":"ok","ambiente":"Staging","versao":"local"}
==> 2/4 POST /api/auth/login
    token obtido (349 caracteres)
==> 3/4 GET /api/consumos (valida acesso ao banco)
    ok
==> 4/4 GET /api/alertas (valida rota protegida por JWT)
    ok
==> SMOKE TEST PASSOU em http://localhost:8081
```

4.4 Smoke test — PRODUÇÃO

```
$ ./scripts/smoke-test.sh http://localhost:8082
==> Aguardando http://localhost:8082/health (timeout 180s)
    API respondeu em 0s
==> 1/4 GET /health
{"status":"ok","ambiente":"Production","versao":"local"}
==> 2/4 POST /api/auth/login
    token obtido (349 caracteres)
==> 3/4 GET /api/consumos (valida acesso ao banco)
    ok
==> 4/4 GET /api/alertas (valida rota protegida por JWT)
    ok
==> SMOKE TEST PASSOU em http://localhost:8082
```

O campo `ambiente` do `/health` comprova qual instância respondeu: **Staging** na 8081 e **Production** na 8082.

Print 4 — Deploy staging

Job deploy-staging com o smoke test passando
docs/prints/05-deploy-staging.png

Print 5 — Deploy produção

Job deploy-production com a aprovação manual registrada
docs/prints/06-deploy-producao.png

Print 6 — Ambientes no ar

/health e Swagger nas portas 8081 e 8082
docs/prints/07-health-staging.png · docs/prints/08-health-producao.png

5. Desafios encontrados e como foram resolvidos

Desafio	O que acontecia	Solução adotada
SQL Server em Apple Silicon	A imagem <code>mssql/server</code> não tem build ARM64; o container não subia na máquina de desenvolvimento.	Troca para <code>mcr.microsoft.com/azure-sql-edge</code> , que tem imagem ARM64 e x86 — a mesma stack roda no Mac e no runner Linux do GitHub.
API subindo antes do banco	As migrations do EF Core rodam no startup e falhavam porque o SQL Server ainda estava inicializando.	<code>healthcheck</code> no serviço <code>mssql</code> testando a porta 1433 + <code>depends_on:</code> <code>condition:</code> <code>service_healthy</code> na API, reforçado por <code>EnableRetryOnFailure()</code> no EF Core.
Migrations quebrando os testes	Os testes usam banco InMemory, que não suporta <code>Database.Migrate()</code> .	Guarda <code>if (!app.Environment.IsEnvironment("Testing"))</code> no <code>Program.cs</code> ; a <code>ApiFactory</code> dos testes sobe com <code>UseEnvironment("Testing")</code> .
Segredos no compose	Senha do banco e chave JWT estavam fixas no <code>docker-compose.yml</code> e no <code>appsettings.json</code> .	Tudo passou a vir de variável de ambiente, com <code>\${DB_PASSWORD:?...}</code> falhando rápido se faltar. Em CI os valores vêm dos secrets do GitHub Environment; <code>.env</code> ficou no <code>.gitignore</code> e só o <code>.env.example</code> é versionado.
Dois ambientes na mesma máquina	Staging e produção colidiam em portas, nomes de container e volumes.	Um único <code>docker-compose.yml</code> parametrizado + um <code>--env-file</code> por ambiente (portas 8081/14331 e 8082/14332) + <code>-p</code> para isolar containers, rede e volume de cada um.
Nome de imagem no GHCR	O push falhava com erro de nome inválido porque <code>github.repository</code> vem com letras maiúsculas e o GHCR exige minúsculas.	Passo que normaliza o nome com expansão do bash — <code>REPO="\${GITHUB_REPOSITORY,,}"</code> — e expõe a tag como output do job.
Healthcheck dentro do container	A imagem <code>aspnet:8.0</code> não traz <code>curl</code> nem <code>wget</code> , então um <code>healthcheck</code> HTTP no compose não funcionaria sem instalar pacote extra e engordar a imagem.	Manter a imagem enxuta e validar a subida de fora, pelo <code>scripts/smoke-test.sh</code> , que é o mesmo script usado no pipeline e localmente.
Evidenciar o ambiente certo	O Swagger só era registrado em <code>Development</code> , e nada na resposta da API dizia em qual ambiente ela	Endpoint <code>/health</code> devolvendo <code>ambiente</code> e <code>versao</code> (o SHA do commit), e Swagger habilitado também em <code>Staging</code> .

	rodava — difícil provar nos prints qual instância respondeu.	
Divergência entre ambientes	Rebuildar a imagem em cada deploy abre espaço para produção receber um binário diferente do que passou em staging.	A imagem é construída uma única vez , marcada com o SHA do commit, e os dois deploys usam <code>--no-build</code> sobre a mesma tag: promoção de artefato em vez de reconstrução.

6. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	☒
Dockerfile funcional	☒
docker-compose.yml ou arquivos Kubernetes	☒
Pipeline com etapas de build, teste e deploy	☒
README.md com instruções e prints	☒
Documentação técnica com evidências (PDF ou PPT)	☒
Deploy realizado nos ambientes staging e produção	☒

Tecnologias utilizadas

Camada	Stack
Aplicação	ASP.NET Core 8 (MVVM), Entity Framework Core 8 + migrations, JWT Bearer, Swagger/OpenAPI
Banco de dados	SQL Server (Azure SQL Edge em container), volume persistente
Frontend	React + Vite + TypeScript
Testes	xUnit, <code>Microsoft.AspNetCore.Mvc.Testing</code> , EF Core InMemory, Bash + curl (smoke test), Postman
Containerização	Docker (multi-stage build), Docker Compose (volumes, redes, variáveis de ambiente)
CI/CD	GitHub Actions, GitHub Environments (segredos + aprovação manual), GitHub Container Registry, cache de build <code>type=gha</code>